Εργασία #1: Πλήρωση Τριγώνων

Βογιατζής Χαρίσιος ΑΕΜ:9192

March 22, 2026

github.com/charisvt/cg-auth-hw1-2026

Abstract

Σκοπός της εργασίας είναι η υλοποίηση αλγορίθμων πλήρωσης τριγώνων με flat shading, Gouraud shading και texture mapping.
Συγκεκριμένα παρουσιάζονται οι: Flat shading, Gouraud shading και Texture shading καθώς και τα αποτελέσματα που παράγουν.

1 Εισαγωγή

Έστω ότι το υπό εξέταση τρίγωνο ορίζεται από τις κορυφές του $p_k = (x_k, y_k) \in \mathbb{Z}^2, k = 0...2$ και τις πλευρές του $e_k, k = 0..2$, τότε μία γραμμή σάρωσης y τέμνει μόνο τις πλευρές p_k για τις οποίες $y_{min} \leq y \leq y_{max}$. Συνεπώς μπορούμε να περιορίσουμε το τρίγωνο ανάμεσα σε δύο γραμμές σάρωσης. Κάθε γραμμή σάρωσης τέμνει ακριβώς 2 πλευρές του τριγώνου με εξαίρεση τις 2 ειδικές περιπτώσεις που ακολουθούν:
1. Γραμμές σάρωσης που αγγίζουν κορυφές
2. Γραμμές σάρωσης που διέρχονται από οριζόντιες πλευρές

2 Flat shading

2.1 Αλγόριθμος

Η συνάρτηση f_shading(img, vertices, vcolors) δέχεται ως ορίσματα έναν καμβά, τις κορυφές των τριγώνων και τα χρώματα που τους αντιστοιχούν και επιστρέφει το χρωματισμένο καμβά.
Για το χρωματισμό του τριγώνου υπολογίζουμε το μέσο όρο των χρωμάτων των κορυφών του χρησιμοποιώντας τη συνάρτηση vector_mean.

```
def vector_mean(v1, v2, v3):
    """
    Calculates the mean of three vectors.
    """
    return (v1 + v2 + v3) / 3
```

Στη συνέχεια υπολογίζουμε τα y_{min} και y_{max} για να περιορίσουμε την περιοχή σάρωσης:

```
ymin = max(int(np.min(vertices[:, 1])), 0)
ymax = min(int(np.max(vertices[:, 1])) + 1, height)
```

Για κάθε γραμμή σάρωσης, βρίσκουμε τα σημεία τομής με τις πλευρές του τριγώνου:

```
for y in range(ymin, ymax+1):
    intersections = []
    for i in range(3):
        p1 = triangle[i]
        p2 = triangle[(i + 1) % 3]
```

```
        y1, y2 = p1[1], p2[1]

        if y1 == y2:
            continue # Skip horizontal edges

        if (y >= y1 and y < y2) or (y >= y2 and y < y1):
            x = p1[0] + (y - y1) * (p2[0] - p1[0]) / (y2 - y1)
            intersections.append(x)
```

Αν βρούμε τουλάχιστον 2 σημεία τομής, τα ταξινομούμε και χρωματίζουμε τα σημεία μεταξύ τους:

```
if len(intersections) < 2:
    continue

intersections.sort()
x_start = max(int(np.ceil(intersections[0])), 0)
x_end = min(int(np.floor(intersections[1])), width - 1)

updated_img[y, x_start:x_end + 1] = flat_color
```

3 Gouraud shading

3.1 Αλγόριθμος

Η συνάρτηση g_shading(img, vertices, vcolors) δέχεται ως ορίσματα έναν καμβά, τις κορυφές των τριγώνων και τα χρώματα των κορυφών και επιστρέφει το χρωματισμένο καμβά.
Αντί για ομοιόμορφο χρωματισμό, το Gouraud shading παρεμβάλλει γραμμικά τα χρώματα των κορυφών σε κάθε pixel του τριγώνου, χρησιμοποιώντας τη συνάρτηση vector_interp.

Ο αλγόριθμος χωρίζεται σε δύο φάσεις:

3.1.1 Φάση 1: Εύρεση σημείων τομής και παρεμβολή χρωμάτων στις ακμές

Για κάθε γραμμή σάρωσης, βρίσκουμε τα σημεία τομής A και B με τις πλευρές του τριγώνου και παρεμβάλουμε τα χρώματα στα σημεία αυτά:

```
for p1, p2, col1, col2 in edges:
    if p1[1] == p2[1]:
        continue # Skip horizontal edges

    if (y >= min(p1[1], p2[1])) and (y <= max(p1[1], p2[1])):
        x = vector_interp(p1, p2, p1[0], p2[0], y, 2)
        c_interp = vector_interp(p1, p2, col1, col2, y, 2)
        intersections.append(x)
        colors_at.append(c_interp)
```

3.1.2 Φάση 2: Παρεμβολή χρώματος ανά pixel

Για κάθε σημείο P=(x, y) στη γραμμή σάρωσης, παρεμβάλουμε το χρώμα μεταξύ των χρωμάτων στα σημεία A και B:

```
for x in range(x_start, x_end + 1):
    color = vector_interp((x_left, y), (x_right, y), c_left, c_right, x, 1)
    img[y, x] = np.clip(color, 0, 1)
```

4 Texture shading

4.1 Αλγόριθμος

Η συνάρτηση t_shading(img, vertices, uv, textImg) δέχεται ως ορίσματα έναν καμβά, τις κορυφές των τριγώνων, τις συντεταγμένες UV και την εικόνα texture και επιστρέφει το χρωματισμένο καμβά. Χρησιμοποιούμε τη βοηθητική συνάρτηση vector_interp για να πραγματοποιήσουμε γραμμική παρεμβολή των συντεταγμένων UV:

```
def vector_interp(p1, p2, V1, V2, coord, dim):
    """
    Linearly interpolates vector V at a point ‘p‘ on the line segment from p1 to p2.
    V can be n-dimensional.
    """
    c1 = p1[dim - 1]
    c2 = p2[dim - 1]

    if c1 == c2:
        return np.array(V1)

    t = (coord - c1) / (c2 - c1)
    return np.array(V1) + t * (np.array(V2) - np.array(V1))
```

Ο αλγόριθμος χωρίζεται σε δύο φάσεις:

4.1.1 Φάση 1: Εύρεση σημείων τομής και συντεταγμένων UV

Για κάθε γραμμή σάρωσης, βρίσκουμε τα σημεία τομής με τις πλευρές του τριγώνου και τις αντίστοιχες συντεταγμένες UV:

```
for p1, p2, uvA, uvB in edges:
    if p1[1] == p2[1]:
        continue # Skip horizontal edges

    if (y >= min(p1[1], p2[1])) and (y <= max(p1[1], p2[1])):
        x = vector_interp(p1, p2, p1[0], p2[0], y, 2)
        uv_interp = vector_interp(p1, p2, uvA, uvB, y, 2)
        intersections.append(x)
        uvs_at.append(uv_interp)
```

4.1.2 Φάση 2: Παρεμβολή συντεταγμένων UV και δειγματοληψία texture

Για κάθε σημείο P=(x, y) στη γραμμή σάρωσης, παρεμβάλουμε τις συντεταγμένες UV και δειγματοληπτούμε το χρώμα από την εικόνα texture:

```
for x in range(x_start, x_end + 1):
    uv_p = vector_interp((x_left, y), (x_right, y), uv_left, uv_right, x, 1)
    tx = int(np.clip(np.round(uv_p[0] * (L - 1)), 0, L - 1))
    ty = int(np.clip(np.round(uv_p[1] * (K - 1)), 0, K - 1))
    img[y, x] = textImg[ty, tx]
```

5 Η συνάρτηση render_img

Η συνάρτηση render_img είναι η κύρια συνάρτηση που χειρίζεται την απόδοση των τριγώνων. Δέχεται ως ορίσματα:

- faces: πίνακας με τις κορυφές των τριγώνων
- vertices: πίνακας με τις συντεταγμένες των κορυφών

- vcolors: πίνακας με τα χρώματα των κορυφών
- uvs: πίνακας με τις συντεταγμένες UV των κορυφών
- depth: πίνακας με το βάθος κάθε κορυφής
- shading: μεταβλητή ελέγχου για επιλογή Flat ('f'), Gouraud ('g') ή Texture ('t') συνάρτησης χρωματισμού
- texImg: η εικόνα texture για το texture mapping

Η συνάρτηση ακολουθεί τα εξής βήματα:

1. Αρχικοποιεί τον καμβά με λευκό χρώμα
2. Ταξινομεί τα τρίγωνα με βάση το μέσο βάθος των κορυφών τους (από πίσω προς τα μπροστά)
3. Για κάθε τρίγωνο, εφαρμόζει την κατάλληλη συνάρτηση χρωματισμού (flat, Gouraud ή texture)
4. Κανονικοποιεί τις τιμές των χρωμάτων στο διάστημα [0, 1]
5. Μετατρέπει την εικόνα σε uint8 για αποθήκευση

```python
def render_img(faces, vertices, vcolors, uvs, depth, shading, texImg=None):
    M = 512
    N = 512
    img = np.ones((M, N, 3), dtype=np.float32)

    depth_flat = depth.flatten()
    avg_depth = np.mean(depth_flat[faces], axis=1)
    sorted_indices = np.argsort(avg_depth)[::-1]

    for idx in sorted_indices:
        face = faces[idx]
        tri_vertices = vertices[face]
        tri_colors = vcolors[face]

        if shading == 'f':
            img = f_shading(img, tri_vertices, tri_colors)
        elif shading == 'g':
            img = g_shading(img, tri_vertices, tri_colors)
        elif shading == 't':
            tri_uvs = uvs[face]
            img = t_shading(img, tri_vertices, tri_uvs, texImg)

    img = np.clip(img, 0, 1)
    return (img * 255).astype(np.uint8)
```

6 Αποτελέσματα

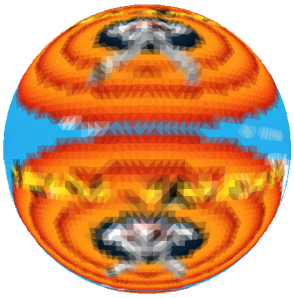

Figure 1: Using Flat shading.

Figure 2: Using Gouraud shading.

Figure 3: Using Texture shading.

7 Κώδικας

Μπορείτε να βρείτε τον κώδικα και τα αποτελέσματα στο Github.